

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a16c0208f4de38372.jsonl`  
- SHA-256: `ad6743437772f989065973c7fdf1e321cf4389d726254114bfba994b69cf6fd`  
- Source modified: `2026-07-02T03:03:23+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T03:02:02.865Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.
A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/khelsutra (avidullu/khelsutra) ? SPA web/ + marketing site/:

```
{  
  "title": "SPA e2e CUJ never runs in CI and the mock engine is hand-synced ? the mock-vs-real drift bug-class already shipped once",  
  "category": "test-gap",  
  "severity": "P2",  
  "file": "khelsutra/.github/workflows/test.yml",  
  "line": 31,  
  "evidence": ".github/workflows/test.yml:31-49: the web job runs only typecheck + test:coverage + build ? the Playwright CUJ specs (web/e2e/ingest.pw.mjs, annotate.pw.mjs, store-medium.pw.mjs against e2e/mock-engine.mjs) are never executed in CI (the workflow's only Playwright job is the site/ layout suite, :51-75). e2e/mock-engine.mjs:5 admits this is deferred: 'used to record the e2e screencast and (later) drive CI CUJ replays'. The cost of hand-syncing the mock is documented in web/src/lib/video.ts:4-6: 'the bug this guards against actually shipped once: the real engine returned id/no-name while the e2e mock faked both, so a green UI was broken live.'",  
  "proposed_fix": "Add a fourth CI job (mirroring the existing layout job's Playwright setup) that builds web/ and runs the e2e CUJ specs against mock-engine.mjs; separately add a thin contract check pinning the mock's response shapes to web/src/api/types.ts (or an engine-side snapshot of _video_summary/capabilities) so mock-vs-real drift fails a build instead of shipping.",  
  "effort": "M",  
  "tracked_in": "e2e/mock-engine.mjs:5 ('later drive CI CUJ replays')"  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T03:02:09.203Z)

```
1     name: test
2     on:
3       push:
4         branches: [master]
5         pull_request:
6
7     jobs:
8       site:
9         runs-on: ubuntu-latest
10        defaults:
11          run:
12            working-directory: site
13        steps:
14          - uses: actions/checkout@v7
15          - uses: actions/setup-node@v6
16            with:
17              node-version: "24" # node:sqlite is unflagged; server.test.js boots the VM
host
18          cache: "npm"
19          cache-dependency-path: site/package-lock.json
20          # jsdom is a TEST-ONLY devDep (the behavioral landing-page UX tests run the real
inline
21          # script in a DOM); the Cloudflare Worker + VM-host RUNTIMES stay dependency-free.
`npm test`
22          # runs ALL paths: Worker+D1 logic (index.test.js), the booted VM host
(server.test.js), the
23          # static content contract (assets.test.js), and the UX behaviour (ux.dom.test.js).
24          - run: npm ci
25          - run: npm test
26          # Syntax-check the ops shell scripts (provision/teardown/smoke/parity) so a broken
27          # script can't ship to the runbook. `bash -n` parses without executing.
28          - name: shell scripts parse-check
29            run: for f in scripts/*.sh; do echo "checking $f"; bash -n "$f"; done
30
31      web:
32        # Operator-console SPA (web/): typecheck + unit tests (with a coverage floor) +
production build
33        # gate the merge. `test:coverage` fails if coverage drops below the thresholds in
vite.config.ts,
34        # so a merge can't silently regress test coverage.
35        runs-on: ubuntu-latest
36        defaults:
37          run:
38            working-directory: web
39        steps:
40          - uses: actions/checkout@v7
41          - uses: actions/setup-node@v6
42            with:
43              node-version: "24"
44              cache: "npm"
45              cache-dependency-path: web/package-lock.json
46          - run: npm ci
47          - run: npm run typecheck
48          - run: npm run test:coverage
49          - run: npm run build
50
51      layout:
52        # Real-browser layout-regression guard. The other suites run in jsdom, which has NO
layout engine
53        # (getBoundingClientRect() is all zeros) and therefore cannot see header overflow /
wrapping /
54        # clipping ? the responsive-nav bug that recurred. This drives a headless Chromium
over the real
```

```

55      # pages at a spread of viewport widths x all 6 locales and asserts the header never
distorts.
56      runs-on: ubuntu-latest
57      defaults:
58        run:
59          working-directory: site
60      steps:
61        - uses: actions/checkout@v7
62        - uses: actions/setup-node@v6
63          with:
64            node-version: "24"
65            cache: "npm"
66            cache-dependency-path: site/package-lock.json
67        - run: npm ci
68        - run: npx playwright install --with-deps chromium
69        - run: npm run test:layout
70      # Cross-locale render guard: drives the real pages in every locale and asserts the
live DOM
71      # matches the catalog (no raw keys / no shim regression / right lang / no console
errors).
72      - run: npm run test:render
73      # Accessibility guard: axe-core WCAG 2.1 A/AA over every page x representative
locales; fails on
74      # any serious/critical violation (jsdom has no contrast/layout engine to catch
these).
75      - run: npm run test:a11y
76

```

3. user (2026-07-02T03:02:10.769Z)

```

ls: cannot access 'C:/Users/avidu/Projects/khelsutra-guru/khelsutra/e2e': No such file or
directory
C:/Users/avidu/Projects/khelsutra-guru/khelsutra/web/e2e:
README.md
annotate.pw.mjs
fixtures
ingest.pw.mjs
mock-engine.mjs
playwright.config.mjs
screencast.pw.mjs
store-medium.pw.mjs
to-mp4.mjs

```

4. user (2026-07-02T03:02:19.273Z)

```

1      // Mock engine ? a dependency-free Node HTTP server implementing the badminton-
highlight-indexer
2      // contract the Studio SPA calls, PLUS serving the built SPA from ./dist. One origin, so
the SPA's
3      // default same-origin `/api/*` calls resolve here exactly as they would behind the
appliance reverse
4      // proxy. This is the hermetic L1 layer from docs/LOCAL_APPLIANCE_ARCHITECTURE.md §8 ?
deterministic,
5      // no GPU/key/Python ? used to record the e2e screencast and (later) drive CI CUJ
replays.
6      //
7      // It is a MOCK: fixtures are canned and the job state machine advances by poll count
(one "running"
8      // poll, then "done") so a screencast shows real progress. The response SHAPES mirror
the verified
9      // engine contract (web/src/api/types.ts + client.ts) so the SPA drives it unmodified.
10     import http from "node:http";
11     import { readFile } from "node:fs/promises";
12     import { existsSync } from "node:fs";
13     import path from "node:path";

```

```

14     import { fileURLToPath } from "node:url";
15
16     const __dirname = path.dirname(fileURLToPath(import.meta.url));
17     const WEB_ROOT = path.resolve(__dirname, ".."); // khelsutra/web
18     const DIST = path.join(WEB_ROOT, "dist");
19     const REEL = path.join(__dirname, "fixtures", "reel.mp4");
20     const PORT = Number(process.env.MOCK_PORT || 8791);
21
22     const MIME = {
23         ".html": "text/html; charset=utf-8", ".js": "text/javascript; charset=utf-8",
24         ".mjs": "text/javascript; charset=utf-8", ".css": "text/css; charset=utf-8",
25         ".json": "application/json; charset=utf-8", ".svg": "image/svg+xml", ".png":
"image/png",
26         ".jpg": "image/jpeg", ".jpeg": "image/jpeg", ".ico": "image/x-icon", ".webp":
"image/webp",
27         ".woff": "font/woff", ".woff2": "font/woff2", ".ttf": "font/ttf", ".mp4": "video/mp4",
28         ".webm": "video/webm", ".map": "application/json",
29     };
30
31     // Deterministic rally fixtures ? the verified play_segments shape (honest fields only).
id 4 sits
32     // below the mock's min_shot_count (5) on purpose, so the SPA's honest floor-warning is
captured.
33     const RALLIES = [
34         { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
35         { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 6 } },
36         { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
37         { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: { shot_count: 4 } },
38         { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
39         { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
40     ];
41
42     const CAPS = {
43         api_version: 1,
44         engine_version: "mock-0.1.0",
45         segmenters: ["gemini", "motion", "wasb_hybrid"],
46         default_segmenter: "gemini",
47         ffmpeg: { ffmpeg: true, ffprobe: true, ffmpeg_path: "/usr/bin/ffmpeg", ffprobe_path:
"/usr/bin/ffprobe" },
48         gpu: { nvidia_smi_present: false },
49         gemini_key_present: true,
50         wasb_weights_present: false,
51         deployment: { profile: "local-appliance", compute_target: "local", cloud_available:
false },
52     // Forward-compatible storage block (STUDIO_MEDIA_LIFECYCLE.md P3/P5). The current SPA
ignores
53     // unknown fields; the medium picker will render from `storage.backends` once P3/P5
land.
54     storage: {
55         default_medium: "local",
56         backends: [
57             { id: "local", kind: "local", label: "This computer", usable:
true, free_gb: 812 },
58             { id: "gcs", kind: "gcs", label: "Google Cloud Storage bucket", usable:
true, free_gb: null },
59             { id: "s3", kind: "s3", label: "Amazon S3 / compatible bucket", usable:
true, free_gb: null },
60             { id: "gdrive", kind: "gdrive", label: "Google Drive folder (mounted)", usable:
true, free_gb: 1900 },

```

5. user (2026-07-02T03:02:19.730Z)

```
1      // Resilient readers for a GET /api/videos row (VideoSummary). Defense-in-depth against
engine drift:
2      // the engine SHOULD return the contract fields (`video_id`, a friendly `match_name`,
`fps`,
3      // `resolution`), but if a build regresses to the raw DB shape (`id`, `filepath`, no
name), the library
4      // + the persistent current-video label still work ? they read through these helpers,
never the raw
5      // field. (The bug this guards against actually shipped once: the real engine returned
`id`/no-name
6      // while the e2e mock faked both, so a "green" UI was broken live.)
7      import type { VideoSummary } from "../api/types";
8
9      /** The MD5 join key ? falls back to the raw `id` column if `video_id` is absent. */
10     export function videoId(v: VideoSummary): string {
11         return v.video_id ?? v.id ?? "";
12     }
13
14     /** Filename stem of a path/URI: `~/My Match 2026.mp4` ? `My Match 2026`. Handles `/'
and `\",
15     * a trailing slash, and a no-extension name; returns "" for empty input. */
16     export function basenameStem(path: string | null | undefined): string {
17         const base =
18             String(path ?? "")
19                 .replace(/[/\\]+$/, "")
20                 .split(/[/\\]/)
21                 .pop() ?? "";
22         const dot = base.lastIndexOf(".");
23         return dot > 0 ? base.slice(0, dot) : base;
24     }
25
26     /** A human name ? the engine's `match_name`, else derived from the stored path/URI,
else "" (the
27     * caller applies its own localized "unnamed"/"your video" fallback). */
28     export function videoName(v: VideoSummary): string {
29         return v.match_name || basenameStem(v.local_path ?? v.filepath);
30     }
```

6. user (2026-07-02T03:02:21.670Z)

```
6f17dc0 chore(ci): bump checkout@v4?v7 and setup-node@v4?v6 (off deprecated Node 20) (#118)
ea7c5fe feat(site): ally audit + fixes + axe CI guard, and localized page titles ? P2 (#107)
fbd7fdd feat(site): finish access-form + footer localization (6 langs) + cross-locale render
guard (#105)
9f77ddc test(site): browser layout-regression CI guard + auto-collapse the nav to fit every
locale (#91)
64076a5 test(web): add a coverage floor (CI-gated) + i18n regression tests for the #77 audit
(#80)
84f436f test(site): behavioral jsdom tests for the submit-feedback + watchable-reveal UX (#38)
23e2104 feat(ops): provider-portable VM provisioning + teardown (destroy -> resume anywhere),
tested live (#16)
aac7e0c feat(web): per-rally MVP P1 ? SPA scaffold + typed engine API client (#14)
939f966 test(site): bake in the both-hosts test harness (Cloudflare + droplet/Node) (#7)
---
f9eea14 web: resilient video-summary readers + show fps in the library (#140)
9223ba9 docs: refresh khelsutra docs for today's storage-track work (P2-P5) (#139)
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
a04a21b docs(tracker): inline the D14 decisions into the P2-P5 rows (#136)
---
```

7. user (2026-07-02T03:02:31.986Z)

test.yml

8. user (2026-07-02T03:02:32.809Z)

```
14: "e2e": "playwright test -c e2e/playwright.config.mjs",
15: "screencast": "vite build && playwright test -c e2e/playwright.config.mjs",
16: "screencast:mp4": "node e2e/to-mp4.mjs"
29: "@playwright/test": "^1.56.0",
```

9. user (2026-07-02T03:02:33.295Z)

```
1 // Playwright config for the Studio screencast / e2e CUJ. Records video of the SPA
(served + API-mocked
2 // by e2e/mock-engine.mjs on ONE origin) and writes the raw .webm into ../../khelsutra-
screencast/_raw
3 // (a sibling, non-git artifacts dir); `npm run screencast` then converts it to a
shareable mp4/gif.
4 // testMatch is `*.pw.mjs` so vitest (which globs src/**/*.test.*) never picks these up.
5 import { defineConfig } from "@playwright/test";
6 import path from "node:path";
7 import { fileURLToPath } from "node:url";
8
9 const __dirname = path.dirname(fileURLToPath(import.meta.url));
10 const PORT = Number(process.env.MOCK_PORT || 8791);
11 // Where the raw .webm lands. Default = the umbrella layout's sibling `khelsutra-
screencast/`
12 // (e2e -> web -> khelsutra -> khelsutra-guru, then khelsutra-screencast/). Override
with the
13 // SCREENCAST_DIR env var on any other machine/layout ? the harness is otherwise self-
contained.
14 const SCREENCAST_DIR = process.env.SCREENCAST_DIR
15 ? path.resolve(process.env.SCREENCAST_DIR)
16 : path.resolve(__dirname, "../../khelsutra-screencast");
17
18 export default defineConfig({
19   testDir: __dirname,
20   testMatch: "**/*.pw.mjs",
21   timeout: 120_000,
22   fullyParallel: false,
23   workers: 1,
24   reporter: [ "list" ],
25   outputDir: path.join(SCREENCAST_DIR, "_raw"),
26   use: {
27     baseURL: `http://127.0.0.1:${PORT}`,
28     viewport: { width: 1280, height: 800 },
29     video: { mode: "on", size: { width: 1280, height: 800 } },
30     launchOptions: { slowMo: 350 }, // a watchable pace for the screencast
31   },
32   webServer: {
33     command: `node ${JSON.stringify(path.join(__dirname, "mock-engine.mjs"))}`,
34     url: `http://127.0.0.1:${PORT}/api/health`,
35     reuseExistingServer: !process.env.CI,
36     timeout: 60_000,
37     env: { MOCK_PORT: String(PORT) },
38   },
39 });
40
```

10. user (2026-07-02T03:02:46.209Z)

khelsutra\docs\STUDIO_MEDIA_LIFECYCLE.md:153:| E2E | **Screencast / e2e harness** ?
single?origin mock engine (`web/e2e/mock-engine.mjs`) + Playwright CUJ (`\*.pw.mjs`) recording
the tool on a local server ? `khelsutra-screencast/`. The **checkpoint?as?screencast**
mechanism; extended per feature. (`npm run screencast`) | khelsutra | ? | no | ? |

```
[#122](https://github.com/avidullu/khelsutra/pull/122) |  
khelsutra\docs\STUDIO_MEDIA_LIFECYCLE.md:156:| P3 | `/api/capabilities` reports **usable storage  
backends** (local always; s3/gcs if SDK+config; gdrive if mounted) via  
`backend/storage/capabilities.py`. Shape = the SPA mock's `storage.backends` block  
(`web/e2e/mock-engine.mjs`). No secrets. | badminton?highlight=indexer | ? | coord | ? |  
[#408](https://github.com/Khelsutra/badminton-highlight-indexer/pull/408) |  
khelsutra\web\e2e\playwright.config.mjs:2:// by e2e/mock-engine.mjs on ONE origin) and writes  
the raw .webm into ../../khelsutra-screencast/_raw  
khelsutra\web\e2e\playwright.config.mjs:33:    command: `node  
${JSON.stringify(path.join(__dirname, "mock-engine.mjs"))}`,  
khelsutra\web\e2e\mock-engine.mjs:177:server.listen(PORT, "127.0.0.1", () => console.log(`[mock-  
engine] http://127.0.0.1:${PORT} serving ${DIST}`));  
khelsutra\web\e2e\README.md:40:- `mock-engine.mjs` ? dependency-free Node server: serves `dist/`  
+ the `/api/*` contract  
khelsutra\web\e2e\README.md:51:As `STUDIO_MEDIA_LIFECYCLE.md` features land, add the endpoint to  
`mock-engine.mjs` and a step to the
```

11. user (2026-07-02T03:03:23.457Z)

Structured output provided successfully